

## Tổng hợp kiến thức Bài 1: Giới thiệu về API và Web Services

### Phần 1: Khái niệm và lịch sử phát triển API

| API là gì?                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | Năm đặc điểm cốt lõi của một API                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | Lịch sử phát triển từ 1998-2015                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p>Là tập hợp quy tắc + giao thức + công cụ để các hệ thống phần mềm giao tiếp: một bên yêu cầu dữ liệu / chức năng, bên kia trả lời theo hợp đồng đã công bố.</p> <p><del>Định nghĩa theo Higgins</del></p> <p>- API được dùng theo 2 nghĩa:</p> <p>+ Nghĩa 1: API của thư viện / framework: Là những hàm, lớp hoặc công cụ có sẵn mà thư viện / framework cung cấp</p> <p>+ Nghĩa 2: API giao tiếp qua mạng: Là cách để các ứng dụng / dịch vụ giao tiếp với nhau qua mạng.</p> | <p>① Contract - First: Có hợp đồng rõ ràng: Thống nhất cách sử dụng API trước khi code, client chỉ cần đọc tài liệu, không cần xem code.</p> <p>② Abstraction: Ẩn cài đặt: Client chỉ cần biết cách sử dụng dịch vụ; cách server xử lý bên trong là chi tiết nội bộ</p> <p>③ Reusability: Có thể tái sử dụng: API cho phép nhiều client dùng lại cùng một chức năng mà không cần sao chép code hoặc sửa phía server</p> <p>④ Language-agnostic: Độc lập với ngôn ngữ, nền tảng: API có thể được sử dụng bởi nhiều ngôn ngữ và nền tảng khác nhau nhờ các định dạng dữ liệu chung như JSON, XML...</p> <p>⑤ Có thể versioning &amp; quan sát được: Versioned &amp; Observable: API có version để cập nhật mà không ảnh hưởng client cũ, cài cơ logs, metrics, trace để theo dõi và quản lý hoạt động</p> | <p>- Trước 1998: Function call nội bộ, CORBA, RPC.</p> <p>- 1998: XML-RPC/SOAP: API dựa trên XML, được sử dụng nhiều trong hệ thống doanh nghiệp.</p> <p>- 2000: REST: Roy Fielding định nghĩa REST, sự ra đời REST phát triển mạnh nhờ HTTP, JSON và được sử dụng rộng rãi cho Web API, mobile API</p> <p>- 2012: GraphQL: Facebook phát triển GraphQL để đáp ứng nhu cầu API linh hoạt cho ứng dụng</p> <p>- 2015: gRPC: Google công bố mã nguồn mở, sử dụng HTTP/2 và Protocol Buffers.</p> <p>Xu hướng phát triển</p> <p>RPC → XML-RPC/SOAP → REST → GraphQL &amp; gRPC</p> <p>→ API ngày càng linh hoạt, đa dạng, phù hợp với nhiều bài ứng dụng</p> |

### Phần 2: Phân biệt API, Web Service & Microservice

| API ≠ Web Service ≠ Microservice                                                                                                                                                                                                                                                                                                            |                                                                                                                                                                                                                                                                     |                                                                                                                                                                                                                                                                                                                     | Web Service chạy trên Client-Server                                                                                                                                                                                                                                                                                                              |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| API                                                                                                                                                                                                                                                                                                                                         | Web Service                                                                                                                                                                                                                                                         | Microservice                                                                                                                                                                                                                                                                                                        |                                                                                                                                                                                                                                                                                                                                                  |
| <p>- Là hợp đồng giao tiếp giữa hai bên, có thể chạy trên bất kỳ môi trường nào</p> <p>- Đặc điểm:</p> <p>+ Là một hợp đồng, không phải một hệ thống.</p> <p>+ Không gắn với công nghệ cụ thể</p> <p>+ Có thể là library-level hoặc network</p> <p>- Ví dụ:</p> <p>+ java.util.List API</p> <p>+ OS/300 OS API</p> <p>+ Github REST API</p> | <p>- Là một API có thể truy cập qua mạng (HTTP), có URL và trả response</p> <p>- Đặc điểm:</p> <p>+ Đi qua mạng (HTTP)</p> <p>+ Bắt buộc có URL / URI</p> <p>+ SOAP + REST là 2 loại chính</p> <p>- Ví dụ:</p> <p>+ api.weather.gov</p> <p>+ graph.facebook.com</p> | <p>- Là một kiến trúc phần mềm chia hệ thống thành nhiều service nhỏ, độc lập</p> <p>- Đặc điểm:</p> <p>+ Một dịch vụ = một chức năng nghiệp vụ</p> <p>+ Giao tiếp với nhau qua API</p> <p>+ Độc lập deploy, scale riêng</p> <p>- Ví dụ:</p> <p>+ orders-service viết bằng Go</p> <p>+ inventory-service Python</p> | <p>- Client và Server giao tiếp với nhau qua mạng.</p> <p>- Client không cần biết cách server xử lý bên trong, chỉ quan tâm tới request và response.</p> <p>- Request: Client gửi → Method + URL + headers + body</p> <p>- Response: Server trả về → Status + header + body</p> <p>→ Client gửi request → Server xử lý → Server trả Response</p> |

### Phần 3: Một số loại API phổ biến

| SOAP: XML & WSDL                                                                                                                                                                                                                                                                                                                                          | REST                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | GraphQL                                                                                                                                                                                                                                                                                                                                                                                     | gRPC                                                                                                                                                                                                                                                                                                                                                                                                                               |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"> <li>- Gọi hàm từ xa bằng XML, có WSDL mô tả API</li> <li>- Quy định chặt chẽ, thường dùng trong hệ thống doanh nghiệp.</li> <li>- Format: XML</li> <li>- Đại diện: Banking legacy, SAP, gov</li> <li>- Ưu điểm: contract chặt chẽ, bảo mật và xử lý lỗi tốt.</li> <li>- Nhược điểm: nặng, phức tạp, khó học</li> </ul> | <ul style="list-style-type: none"> <li>- Là phong cách kiến trúc, không phải giao thức</li> <li>- 6 ràng buộc của một hệ thống: Client-Server, Stateless, Cacheable, Uniform Interface, Layered System, Code-on-demand</li> <li>- Format: JSON, XML, YAML</li> <li>- Đại diện: github API, Stripe API</li> <li>- Ưu điểm: đơn giản, dễ debug</li> <li>- Nhược điểm: lấy theo dữ liệu, lấy theo dữ liệu, n+1 problem</li> </ul>                                                                                                                                                                                                                                                                                                                                                                         | <ul style="list-style-type: none"> <li>- Dùng một endpoint duy nhất cho mọi loại truy vấn. Client tự chỉ định dữ liệu cần lấy, server trả đúng dữ liệu đó</li> <li>- Format: JSON</li> <li>- Đại diện: Github GraphQL, Shopify</li> <li>- Ưu điểm: quản lý theo hoặc theo dữ liệu, phù hợp với dữ liệu phức tạp</li> <li>- Nhược điểm: caching và up load file phức tạp hơn REST</li> </ul> | <ul style="list-style-type: none"> <li>- Client gọi hàm trên server, dùng HTTP/2 + Protobuf.</li> <li>- Thường dùng cho các microservice</li> <li>- Giao tiếp nhanh và chính xác</li> <li>- Format: Protobuf (binary)</li> <li>- Đại diện: Service-to-service trong Google, Netflix, ...</li> <li>- Ưu điểm: nhanh, dữ liệu nhỏ, contract chặt chẽ, có thể sinh code tự động</li> <li>- Nhược điểm: khó đọc → khó debug</li> </ul> |
| <p>Khi nào chọn loại API nào?</p>                                                                                                                                                                                                                                                                                                                         | <ul style="list-style-type: none"> <li>- REST: Phù hợp với web, Mobile, Public API. Cách sử dụng đơn giản, thường truyền dữ liệu bằng JSON và phù hợp với các hệ thống thông thường.</li> <li>- GraphQL: Phù hợp khi có nhiều loại Client và mỗi client cần những phần dữ liệu khác nhau, client có thể tự chọn các trường dữ liệu cần lấy.</li> <li>- gRPC: Phù hợp cho giao tiếp giữa các microservice, đặc biệt khi cần tốc độ cao, truyền dữ liệu hiệu quả và contract chặt chẽ</li> <li>- SOAP: Phù hợp với các hệ thống doanh nghiệp / legacy đã sử dụng SOAP và WSDL. Có contract chặt chẽ, nhưng phức tạp và nặng hơn REST</li> </ul> <p>⇒ Nguyên tắc lựa chọn: Việc chọn API phụ thuộc vào cơ sở sử dụng API, client cần dữ liệu gì, yêu cầu về contract và tình trạng hệ thống hiện tại.</p> |                                                                                                                                                                                                                                                                                                                                                                                             |                                                                                                                                                                                                                                                                                                                                                                                                                                    |

### Phần 4: Thực hành với Flask: REST + JSON

- Các lỗi thường gặp khi mới viết API:
1. Trả 200 cho mọi thứ, kể cả lỗi  
Body chứa {"success": false} nhưng status=200 → client không retry, monitoring không alert, khó tách lỗi
  2. Dùng từ trong URL:  
/get User /42, /create Order, /delete Account → đây là phong cách RPC, không phải REST phải là GET /user/42, ... mỗi đưng
  3. In-everything validation hoặc không validate gì cả  
Kiểm tra quá ít (tin tưởng limit) → Key Error 500  
Kiểm tra mọi thứ (mỗi hàm validate riêng) → khó tái sử dụng
  4. Trộn business logic với HTTP layer  
Toàn bộ tính toán nằm trong route → unittest phải qua HTTP,
  5. Hard-core URL /secret trong code, không log gì cả  
gh repo có API Key → phải clone từ env. không log request body có password